Initial Placement for Fruchterman–Reingold Force Model with Coordinate Newton Direction

Hiroki Hamaguchi Naoki Marumo Akiko Takeda

May 17, 2026

Abstract

The Fruchterman–Reingold (FR) force model is widely used in force-directed graph drawing, and multilevel approaches such as sfdp in Graphviz scale these methods effectively. A crucial step in multilevel schemes is refinement, which improves the graph layout typically performed with the FR algorithm. For this refinement, optimization-based methods, such as L-BFGS, often yield better layouts by exploiting past gradient information. However, they suffer from high per-iteration costs for large graphs and difficulty in handling highly twisted graphs.

In this research, we propose a new initial placement based on the stochastic coordinate descent to accelerate the optimization process. We first reformulate the problem as a discrete optimization problem using a hexagonal lattice and then iteratively update a randomly selected vertex along the coordinate Newton direction with low per-iteration costs.

We demonstrate the effectiveness of our method through numerical experiments, showing that our initial placement leads to faster convergence and higher-quality layouts compared to naive optimization approaches. We also discuss how the coordinate Newton direction can be applied to other graph-related problems from an optimization perspective.

1 Introduction

Graph drawing is a fundamental task in information visualization, and force-directed methods are among the most widely used approaches. In these methods, vertices are treated as particles subject to forces, and the resulting equilibrium layouts are regarded as desirable visualizations. Among established force models [8, 21], the Fruchterman–Reingold (FR) force model [10] is particularly prominent for its intuitive formulation, flexibility, and simplicity. It is implemented in libraries such as NetworkX [15], Graphviz [9], and igraph [6].

Obtaining the equilibrium layout of the force models has been a subject of extensive research. The FR algorithm [10] is an original iterative method that simulates the forces to find an equilibrium, but it struggles with its scalability. Its convergence becomes prohibitively slow on large graphs, prompting the development of multilevel coarsen-refine schemes such as the Scalable Force-Directed Placement (sfdp) method in Graphviz [9, 20]. While these frameworks extend the applicability, their refinement stage still relies heavily on the FR algorithm, leaving room for improvement.

The critical issue in the refinement step is that the twist slows down the force simulation. The term twist refers to unnecessary folded and tangled structures in the visualized graph [5, 30]. The results in Fig. 1 highlight these twist issues. Even if a graph has a

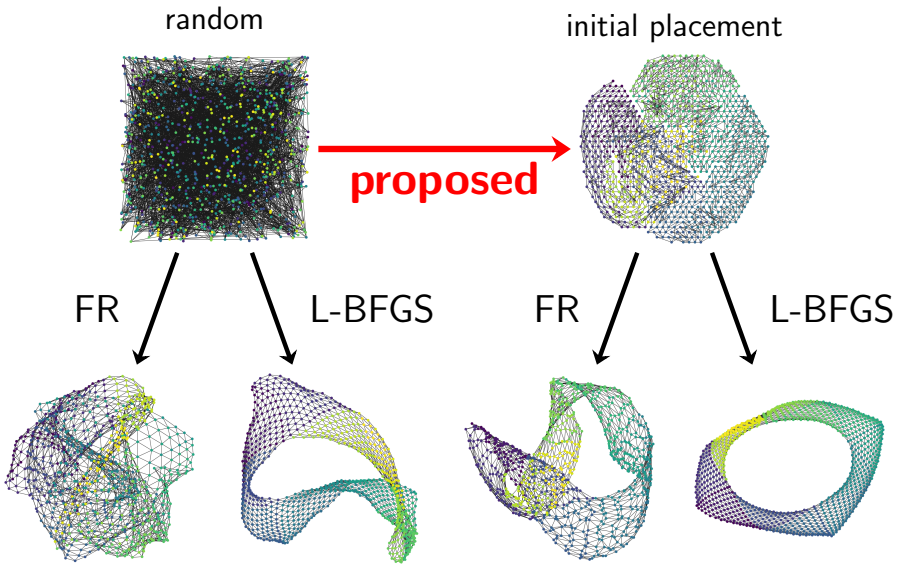

Figure 1: Comparison of the algorithms for the `jagmesh1`.

simple structure, twists often occur, which cancel out the forces and lead to stagnation and suboptimal visualization outcomes.

To address the twist problem and obtain better equilibrium layouts, we utilize continuous optimization techniques. The L-BFGS algorithm [23] is a general purpose optimization algorithm and more practical approach [19], achieving better results for several force models including the FR force model and the Kamada–Kawai (KK) model [21]. While L-BFGS algorithm can partially address the twist problem, it can require many iterations with high per-iteration cost. It may fail to achieve optimal visualization within a limited number of iterations as shown in Fig. 1. A Stochastic Gradient Descent (SGD)-based method, which updates the positions of two vertices at a time based on a single edge, has recently been proposed for the KK layout. [21, 34]. While it is highly regarded, applying it to the force model in the FR algorithm is challenging due to its relatively complex structure. Refer to Sec. 6 for more details.

Pre-processing to find a better initial placement is another optimization-based approach to solve the twist problem. A pre-processing step with Simulated Annealing (SA) is known to be effective since SA, the optimization method, can avoid getting stuck in local optima and leads to a better visualization combined with the FR algorithm [13]. Still, as discussed in Sec. 3.3, this work focuses only on unweighted, simple-structured, and small-scale graphs.

In this paper, we propose a new initial placement for the force model in the FR algorithm. We provide an initial placement with fewer twists than random placement within a short time, accelerating the subsequent optimization process. We can use both FR and L-BFGS algorithms to obtain the final placement. This work extends the applicability of the initial placement idea to larger-scale, weighted, and complicated structured graphs. To achieve this, we optimize the position of vertices one by one with the coordinate Newton direction, leveraging the inherent structure and the sparsity of graphs. We also demonstrate its effectiveness through various experiments.

The remainder of the paper is organized as follows.

- In Sec. 2, we provide the technical definitions of the FR force model.
- In Sec. 3, we review some related work.

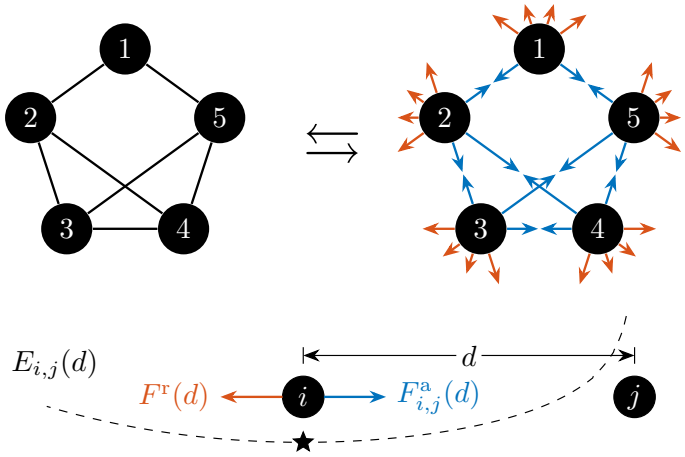

Figure 2: In the model, forces act on every pair of vertices. Forces $F^{\mathrm{a}}_{i,j}(d)$ and $F^{\mathrm{r}}(d)$ work between vertices i and j. The equilibrium of them is achieved at $d = k/\sqrt[3]{a_{i,j}}$, which equals k when $a_{i,j} = 1$.

- In Sec. 4, we present our initial placement algorithm.
- In Sec. 5, we report experimental comparisons.
- In Sec. 6, we discuss our approach and its potential applications.
- In Sec. 7, we conclude the paper and discuss future work.

2 Preliminaries

Fruchterman and Reingold [10] proposed the FR algorithm with its force model for graph drawing based on the physical analogy of the system of particles. Note that we distinguish between the FR algorithm and the FR model. The former is an iterative method that simulates the forces, while the latter is a mathematical formulation of the forces. Through the simulation of these forces, the FR algorithm seeks equilibrium positions. In contrast to this ordinary approach, energy-based methods seek equilibrium. A local minimum of the energy function Φ yields an equilibrium position since $\nabla\Phi(X)$ corresponds to the forces. In this sense, the force-based and energy-based perspectives are equivalent as both capture equilibrium, illustrated in Fig. 2.

Let us formally define the optimization problem for obtaining the equilibrium layout. Let $G = (V, E)$ be an undirected graph with vertex set $V = \{1, \dots, n\}$ and edge set E. Each edge $\{i, j\} \in E$ has positive weight $a_{i,j} = a_{j,i} \in \mathbb{R}$. For convenience, we set $a_{i,j} = 0$ for $\{i, j\} \notin E$. The force model in the FR algorithm assumes forces between vertices. For vertices i and j with distance $d > 0$, an attractive force $F^{\mathrm{a}}_{i,j}(d)$ and a repulsive force $F^{\mathrm{r}}(d)$ are defined as

$$F^{\mathrm{a}}_{i,j}(d) \coloneqq \frac{a_{i,j} d^2}{k}, \quad F^{\mathrm{r}}(d) \coloneqq -\frac{k^2}{d},$$

where $k > 0$ is a constant parameter, often set to $1/\sqrt{n}$ [10]. The scalar potentials of these forces [19] are given by

$$\Phi^{\mathrm{a}}_{i,j}(d) \coloneqq \int_0^d F^{\mathrm{a}}_{i,j}(r)\,\mathrm{d}r = \frac{a_{i,j} d^3}{3k}, \quad \Phi^{\mathrm{r}}(d) \coloneqq \int_1^d F^{\mathrm{r}}(r)\,\mathrm{d}r = -k^2 \log d,$$
$$\Phi_{i,j}(d) \coloneqq \Phi^{\mathrm{a}}_{i,j}(d) + \Phi^{\mathrm{r}}(d).$$

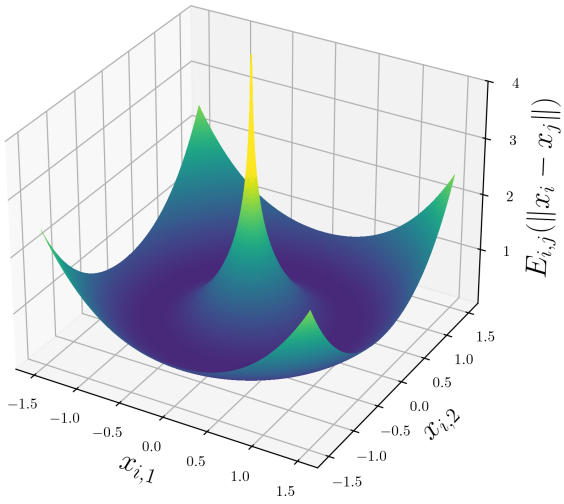

Figure 3: The energy function $\Phi_{i,j}(\|x_i - x_j\|)$ for $x_i = (x_{i,1}, x_{i,2})$, $x_j = (0, 0)$, $a_{i,j} = 1$, and $k = 1$.

For simplicity, we define $\Phi_{i,j}(0) = \infty$.[1] Let $\|\cdot\|$ denote the Euclidean norm in $\mathbb{R}^2$. Then, the problem is to minimize the energy with $X \coloneqq (x_1, \ldots, x_n) \in \mathbb{R}^{2\times n}$:

$$\underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad \Phi(X) \coloneqq \sum_{\substack{(i,j)\in V^2 \\ i\neq j}} \Phi_{i,j}(\|x_i - x_j\|). \tag{1}$$

Note that the summation is taken over ordered pairs (i, j). For undirected graphs, this includes both (i, j) and (j, i) and therefore only scales the objective value by a constant factor. We keep this form since it extends naturally to directed or asymmetric weights.

While the energy function $\Phi_{i,j}(d)$ is convex and minimized when $d = k/\sqrt[3]{a_{i,j}}$ for a positive $a_{i,j}$, a function $x_i \mapsto \Phi_{i,j}(\|x_i - x_j\|)$ is non-convex for a fixed x_j. Additionally, $\Phi_{i,j}$ is not Lipschitz continuous near $d = 0$, as illustrated in Fig. 3. These properties highlight the difficulty of the optimization problem.

3 Related Work

We next summarize the existing literature relevant to our work.

3.1 Optimization Approach

Optimization-based methods for graph drawing have been actively investigated. Representative approaches include GPU acceleration [12], machine learning techniques [28], and multi-objective formulations [1]. Among optimization-based approaches, our attention is on the underlying optimization algorithms itself. In this section, we review the L-BFGS algorithm.

The L-BFGS algorithm, a renowned and widely used optimization method, consistently outperforms the original FR algorithm in speed and layout quality [19]. It uses the function value and the gradient to find the local optima. We define the sum of terms associated

[1] We can also use $-k^2 \log(d + \epsilon^{\mathrm{r}})$ for $\Phi^{\mathrm{r}}(d)$ to prevent divergence, where ϵ^{r} is constant.

with the vertex x_i as Φ_i, which is explicitly given by

$$\Phi_i(x_i) \coloneqq \sum_{j \in V \setminus \{i\}} \Big(\Phi_{i,j}(\|x_i - x_j\|) + \Phi_{j,i}(\|x_j - x_i\|) \Big).$$

The i-th column of the gradient $\nabla\Phi(X) \in \mathbb{R}^{2\times n}$ is

$$\begin{aligned}(\nabla\Phi(X))_i \coloneqq \nabla\Phi_i(x_i) &= \sum_{j \in V \setminus \{i\}} \left(\frac{(a_{i,j} + a_{j,i})\|x_i - x_j\|}{k} - \frac{2k^2}{\|x_i - x_j\|^2} \right) (x_i - x_j) \\ &= 2 \sum_{j \in V \setminus \{i\}} \left(\frac{a_{i,j}\|x_i - x_j\|}{k} - \frac{k^2}{\|x_i - x_j\|^2} \right) (x_i - x_j).\end{aligned}$$

The graph layout can be obtained as the result of optimization using the L-BFGS solver with the gradient described above. A common choice for the initial solution is a random placement of nodes within the bounding box $\{(x_i^{(1)}, x_i^{(2)}) \mid 0 \le x_i^{(1)} \le 1, 0 \le x_i^{(2)} \le 1\}$ [31]. However, when the input is highly twisted or otherwise ill-conditioned, L-BFGS can suffer from large per-iteration costs: each iteration operates on the full set of coordinates, which limits its ability to untangle local geometric defects.

todo. describe here.

3.2 Multilevel Approach

Multilevel approaches such as sfdp [20] in Graphviz are important prior work for obtaining the equilibrium layout of the force model, i.e., solving the problem (1). These methods use hierarchical coarsening and refinement to achieve efficient layout computation for large graphs.

Our work is complementary to such multilevel methods. Optimization techniques such as L-BFGS can be directly applied to the refinement stages within multilevel pipelines. Since the approaches proposed in this study can be incorporated into multilevel frameworks with appropriate modifications, our primary objective is to quantify and enhance the refinement procedure itself. Therefore, we deliberately exclude coarsening and other approximation steps from the present analysis. For further details on multilevel approaches, the reader is referred to [2, 17, 32].

3.3 Initial Placement Approach

A preprocessing step with Simulated Annealing (SA) is known to be effective [13] since SA can avoid getting stuck in local optima and leads to better visualization, combined with the FR algorithm. Let

$$Q^{\text{circle}} \coloneqq \{(\cos(2\pi i/n), \sin(2\pi i/n)) \mid 1 \le i \le n\}$$

be the points on a unit circle in $\mathbb{R}^2$. For an unweighted graph G, let E_2 be a set of vertex pairs with a shortest path distance equal to 2. Let $\angle(a, b)$ denote the angle between the lines from the origin to the points a and b, measured in the interval $(-\pi, \pi]$. This study [13] defines the problem for the preprocessing of graph drawing as follows:

$$\begin{aligned} \underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad & \sum_{\{i,j\} \in E \cup E_2} |\angle(x_i, x_j)|, \\ \text{subject to} \quad & x_i \in Q^{\text{circle}} \quad \text{for } 1 \le i \le n, \\ & x_i \ne x_j \quad \text{for } 1 \le i < j \le n. \end{aligned} \tag{2}$$

The problem (2) is a discrete optimization problem where the placement is limited to Q^{circle} and uses angles, not the function Φ in the problem (1). This study obtains a faster and better visualization by setting the result of Simulated Annealing (SA) for the problem (2) as an initial placement for the FR algorithm.

Still, the following limitations remain:

- The target graphs are restricted to unweighted ones.
- The layout is confined to a simple circle, which could be ineffective for complex structured graphs.
- The neighborhood in the SA is the random swapping of two vertices, making the optimization process inefficient for large-scale graphs.
- $|E_2|$ could be $\Theta(|V|^2)$, unable to leverage the sparsity of graphs if it exists.

We address these limitations, and our study is an extension of this work.

4 Proposed Algorithm

With the prior studies in Sec. 3.1, this section proposes Algorithm 1, a new initial placement algorithm for the problem (1). The algorithm solves a discrete optimization problem with stochastic coordinate descent to find an initial placement with fewer twists. This algorithm leverages the properties of the objective function to determine the initial layout quickly. By combining it with optimization methods like the L-BFGS algorithm, it enhances the overall convergence speed of graph drawing through optimization.

4.1 Newton Direction and Coordinate Newton Direction

Consider a strictly convex function $g\colon \mathbb{R}^n \to \mathbb{R}$ at x_0. The Newton direction $d = -\nabla^2 g(x_0)^{-1}\nabla g(x_0)$ is an optimal direction for the second order approximation of g:

$$g(x_0) + \nabla g(x_0)^\top (x - x_0) + \frac{1}{2}(x - x_0)^\top \nabla^2 g(x_0)(x - x_0).$$

$x = x_0 + d$ is the minimizer of this approximation. Note that the Hessian matrix $\nabla^2 g(x_0)$ is positive definite since g is strictly convex. Although the Newton direction is essential in various iterative methods, it requires the computation of the inverse Hessian $\nabla^2 g(x_0)^{-1} \in \mathbb{R}^{n\times n}$, posing a high computational cost for large-scale problems.

Still, we can leverage the concept of the Newton direction in a different manner, the coordinate Newton direction. Instead of computing the inverse Hessian $\nabla^2 g(x_0)^{-1}$ in the entire variable space $\mathbb{R}^n$, we restrict the variable x to its coordinate block x_i with fewer dimensions, and compute $\nabla^2 g_i(x_i)^{-1}\nabla g_i(x_i)$ where g_i is a restricted function of g to x_i. Since the coordinate Newton direction computation is much cheaper than that of the Newton direction, we can repeat this procedure many times. In general, this idea is known as stochastic coordinate descent [33] or Randomized Subspace Newton [14] in a broader context.

In particular, this coordinate Newton direction has an apparent natural affinity to the problem (1). We can compute the coordinate Newton direction by taking the position x_i of the vertex i as the coordinate block. Although directly applying this idea to the problem (1) is challenging, as we will discuss in Sec. 6, we leverage this coordinate Newton direction to propose our algorithm.

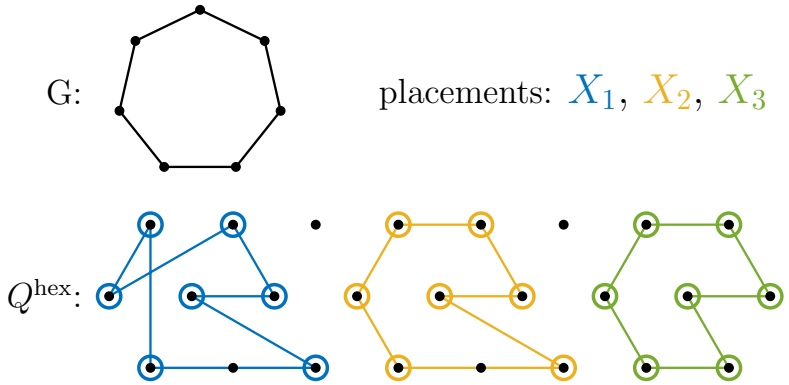

Figure 4: Concept of Q. The assignment from V to a discrete point placement Q, especially a hexagonal lattice Q^{hex}. Apparently, among the three placements, the right one is the best placement for the problem (3).

4.2 Discrete Optimization Problem for Initial Placement

Even at the expense of accuracy, obtaining an approximate solution quickly is crucial for the initial placement. To obtain it, we simplify the problem (1) into a more manageable and well-behaved discrete optimization problem:

$$\begin{aligned} \underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad & \Phi^{\mathrm{a}}(X) \coloneqq \sum_{\{i,j\}\in E} \frac{a_{i,j}\|x_i - x_j\|^3}{3k}, \\ \text{subject to} \quad & x_i \in Q^{\mathrm{hex}} \quad \text{for } 1 \le i \le n, \\ & x_i \neq x_j \quad \text{for } 1 \le i < j \le n, \end{aligned} \tag{3}$$

where

$$Q^{\mathrm{hex}} \coloneqq \left\{ \left(q + \frac{1}{2}r, \frac{\sqrt{3}}{2}r \right) \,\middle|\, q \in \mathbb{Z},\ r \in \mathbb{Z} \right\}. \tag{4}$$

First, the problem (1) is equivalent to the following:

$$\underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad \sum_{\{i,j\}\in E} \frac{a_{i,j}\|x_i - x_j\|^3}{3k} - \sum_{i<j} k^2 \log \|x_i - x_j\|.$$

This formulation separates the $\mathcal{O}(|E|)$ terms from $\Phi^{\mathrm{a}}_{i,j}$ and the $\mathcal{O}(|V|^2)$ terms from Φ^{r}.

Here, following previous research mentioned in Sec. 3.3, we fix the possible positions x_i of each vertex i to a discrete set of points Q, where $|Q| \ge |V|$. It means that we consider the following:

$$\begin{aligned} \underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad & \sum_{\{i,j\}\in E} \frac{a_{i,j}\|x_i - x_j\|^3}{3k} - \sum_{i<j} k^2 \log \|x_i - x_j\|, \\ \text{subject to} \quad & x_i \in Q \quad \text{for } 1 \le i \le n, \\ & x_i \neq x_j \quad \text{for } 1 \le i < j \le n. \end{aligned}$$

The goal is to simplify the objective function and choose Q appropriately to derive a well-simplified problem.

Due to the sparsity of many practical graphs, $|E| \ll |V|^2$ holds. To leverage this sparsity for simplification, we want to drop the second term that arises from the repulsive energy Φ^r:

$$-\sum_{i<j} k^2 \log \|x_i - x_j\|.$$

We impose a condition on Q such that $\|q_i - q_j\| \geq \epsilon$ for all $q_i, q_j \in Q$ with $q_i \neq q_j$. In this case, the term above is negligible. Since $\Phi^{\mathrm{r}}(d) = -k^2 \log d$ is a convex function such that it decreases monotonically concerning d, for sufficiently large d, the value of $-k^2 \log d$ does not vary excessively. For too small d, we can prevent the divergence of the energy function by setting ϵ. Thus, under this condition, we drop the second term and derive the problem as:

$$\begin{aligned} \underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad & \sum_{\{i,j\}\in E} \frac{a_{i,j}\|x_i - x_j\|^3}{3k}, \\ \text{subject to} \quad & x_i \in Q \quad \text{for } 1 \leq i \leq n, \\ & x_i \neq x_j \quad \text{for } 1 \leq i < j \leq n. \end{aligned}$$

This means that we skip to consider the $\mathcal{O}(|V|^2)$ pairs (all pairs of $\{i, j\}$ with $1 \leq i < j \leq n$) by fixing the possible point placement in advance, reducing the computational complexity to $\mathcal{O}(|E|)$ and thus offering significant speedup. See Fig. 4 for a visual explanation.

We can consider various placements for the Q, including Q^{circle} [13]. In this study, we adopt a hexagonal lattice Q^{hex} [22, 26] defined by Eq. (4) (see Fig. 4). When minimizing the objective function that arises from the attractive energy Φ^{a}, it is advantageous for the points to cluster as closely as possible. In this context, the hexagonal lattice is known for its densest packing structure in space with the least distance ϵ between points [4] and offers computational simplicity. Thus, Q^{hex} is a suitable choice, and we have derived the problem (3).

4.3 Newton Direction for Discrete Optimization

Next, we solve the discrete optimization problem (3). Although this problem is challenging to solve just using simple methods, the coordinate Newton direction of a randomly selected vertex i provides significant insights as mentioned in Sec. 4.1. Therefore, we can offer a high-quality solution to the problem. Let the restricted objective function $\Phi_i^{\mathrm{a}}(x_i)$ be

$$\Phi_i^{\mathrm{a}}(x_i) \coloneqq \sum_{j\in V\setminus\{i\}} \frac{a_{i,j}\|x_i - x_j\|^3}{3k}.$$

Its gradient and Hessian matrix are

$$\begin{aligned} \nabla\Phi_i^{\mathrm{a}}(x_i) &= \sum_{j\in V\setminus\{i\}} \frac{a_{i,j}\|x_i - x_j\|}{k}(x_i - x_j), \\ \nabla^2\Phi_i^{\mathrm{a}}(x_i) &= \sum_{j\in V\setminus\{i\}} \frac{a_{i,j}\|x_i - x_j\|}{k}\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \sum_{j\in V\setminus\{i\}} \frac{a_{i,j}}{k\|x_i - x_j\|}(x_i - x_j)(x_i - x_j)^\top. \end{aligned}$$

This means Φ_i^{a} is strictly convex, assuring the Hessian matrix $\nabla^2\Phi_i^{\mathrm{a}}(x_i)$ is positive definite. This is a large difference from the functions $\Phi(X)$ in the problem (1) and $\Phi^{\mathrm{a}}(X)$ in the problem (3), which are non-convex.

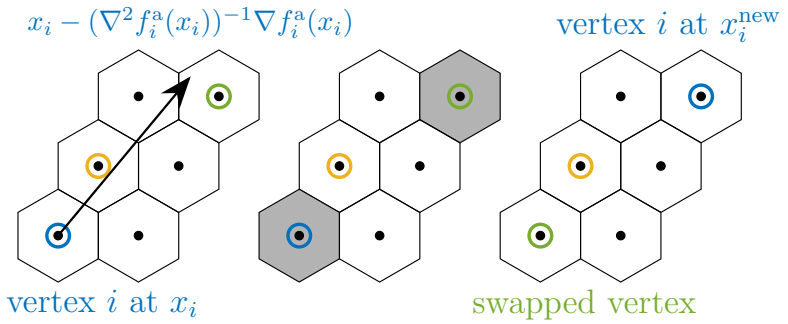

Figure 5: One iteration of the proposed algorithm. Step 1. Compute the coordinate Newton direction (blue). Step 2. Decide x_i^{new} by rounding the direction and adding a random vector. Step 3. Move the vertex and swap the vertices if necessary (swap blue and green vertices).

The ordinary updated rule with the coordinate Newton direction is

$$x_i^{\mathrm{new}} \leftarrow x_i - \nabla^2 \Phi_i^{\mathrm{a}}(x_i)^{-1} \nabla \Phi_i^{\mathrm{a}}(x_i).$$

x_i^{new} may not be in the hexagonal lattice Q^{hex} in the problem (3). Thus, we project this x_i^{new} onto the nearest point in Q^{hex}. We also empirically found that adding a random noise vector to the Newton direction is effective for the optimization process, a strategy similar to the SA in Sec. 3.3. This randomness can help to escape from local minima and to explore the solution space more effectively. In conclusion, the updated rule for the vertex i is

$$x_i^{\mathrm{new}} \leftarrow \mathrm{round}\big(x_i - \nabla^2 \Phi_i^{\mathrm{a}}(x_i)^{-1} \nabla \Phi_i^{\mathrm{a}}(x_i) + t \cdot r\big),$$

where $\mathrm{round}(\hat{x})$ denotes the operation assigning $\hat{x}$ to the nearest point in the hexagonal lattice Q^{hex}, r is a random vector with a unit norm, and the parameter t is gradually decreased as the iterations proceed and is designed to converge to zero eventually.

If there is a vertex j such that $x_j = x_i^{\mathrm{new}}$, we swap the positions x_i and x_j to satisfy the condition $x_i \neq x_j$. Otherwise, we just update x_i to x_i^{new}. Refer to Fig. 5 for a visual explanation. Repeating this procedure yields an approximate solution to the problem (3).

4.4 Optimal Scaling

The obtained solution could be too small or too large since we did not care about the scale ϵ. Thus, as the final step, we rescale the placement to improve it.

Let us formulate the optimization problem for the scaling factor $s > 0$. For an initial placement $X = (x_1, \ldots, x_n)$, we scale it as $x_i \leftarrow s x_i$ for all i. The problem of minimizing $\Phi(X)$ through scaling is equivalent to minimizing $\phi(s)$ defined by

$$\phi(s) := \sum_{\{i,j\} \in E} \frac{a_{i,j} (s \|x_i - x_j\|)^3}{3k} - k^2 \sum_{i<j} \log(s \|x_i - x_j\|),$$

$$\phi'(s) = \sum_{\{i,j\} \in E} \frac{a_{i,j} \|x_i - x_j\|^3}{k} s^2 - \frac{k^2 n(n-1)}{2s}.$$

The function $\phi(s)$ is convex, and the optimal scaling factor s^* satisfies $\phi'(s^*) = 0$, which yields

$$s^* = \left(\frac{k^3 n(n-1)}{2 \sum_{\{i,j\} \in E} a_{i,j} \|x_i - x_j\|^3} \right)^{1/3}. \tag{5}$$

This value can be computed in $\mathcal{O}(|E|)$ complexity, enabling us to obtain a better initial placement for the problem (1).

Notably, the optimal solution to the problem (3) is invariant under scaling. Thus, we do not have to care about the scale factor for the hexagonal lattice Q^{hex} as far as we scale the obtained placement by s^*.

Algorithm 1: Proposed algorithm for the initial placement

Input: Graph $G = (V, E)$, Weight $(a_{i,j})_{\{i,j\}\in E}$, Parameters $N_{\mathrm{iter}}^{\mathsf{CN}} \in \mathbb{N}$, and $t_0 > 0$
Output: Initial placement $X = (x_1, \dots, x_n)$

$t \leftarrow t_0$;
Sample $x_i \in Q^{\mathrm{hex}}$ for all $i \in V$ without replacement;
for $m \leftarrow 0$ **to** $N_{\mathrm{iter}}^{\mathsf{CN}}$ **do**
  Select vertex $i \in V$ randomly;
  Draw $r \in \mathbb{R}^2$ randomly from a unit circle;
  $x_i^{\mathrm{new}} \leftarrow \mathrm{round}(x_i - \nabla^2\Phi_i^{\mathrm{a}}(x_i)^{-1}\nabla\Phi_i^{\mathrm{a}}(x_i) + t \cdot r)$;
  if $\exists j \in V$ *such that* $x_j = x_i^{\mathrm{new}}$ **then**
    $x_j \leftarrow x_i$;
  $x_i \leftarrow x_i^{\mathrm{new}}$;
  $t \leftarrow t - t_0/N_{\mathrm{iter}}^{\mathsf{CN}}$;
$x_i \leftarrow s^* x_i$ for all $i \in V$ with s^* given by Eq. (5);
return X

4.5 Alternative Approach

To end this section, we explain an alternative approach to the problem (3). We can also solve the problem by updating all vertices simultaneously, not one by one. It means moving all the points $\{x_i\}_{i\in V}$ to arbitrary positions and then assigning all these $|V|$ points to the nearest points on Q^{hex}. The optimal assignment for $\{x_i\}_{i\in V}$ to Q^{hex} is computable by a Hungarian algorithm in $\mathcal{O}(|V|^3)$ time or some heuristic.

4.6 Application to Other Force Models

We briefly discuss and explore the potential applicability of stochastic coordinate descent to a broader range of problems. Although we utilized the coordinate Newton direction only for the optimization problem (1), we can see that its application is not necessarily limited to the FR force model, the problem (1).

Recall that Our optimization problem (1) can be rewritten as

$$\underset{X\,\in\,\mathbb{R}^{2\times n}}{\text{minimize}} \quad \Phi_{\mathrm{FR}}(X) = \sum_{(i,j)\in E} \frac{a_{i,j}\|x_i - x_j\|^3}{3k} - \sum_{\substack{(i,j)\in V^2\\ i\neq j}} k^2 \log\left(\epsilon^{\mathrm{r}} + \|x_i - x_j\|\right).$$

Similar objective functions arise from other force models, such as ... are as follows:

$$\underset{X\,\in\,\mathbb{R}^{2\times n}}{\text{minimize}} \quad \Phi_{\mathrm{HC}}(X) = \sum_{(i,j)\in E\wedge j>i} \frac{k_1(\|x_i - x_j\| - a_{ij})^2}{2} + \sum_{\substack{(i,j)\in V^2\\ i\neq j}} \frac{k_2}{\|x_i - x_j\| + \epsilon^{\mathrm{r}}}$$

$$\underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad \Phi_{\text{Eades}}(X) = \sum_{(i,j)\in E} k_1 \|x_i - x_j\| \left(\log \frac{\|x_i - x_j\|}{a_{ij}} - 1 \right) + \sum_{\substack{(i,j)\in V^2 \\ i\neq j}} \frac{k_2}{\|x_i - x_j\| + \epsilon^{\text{r}}}$$

Clearly, these objective functions are also non-convex and have a similar structure to $\Phi_{\text{FR}}(X)$, which consists of a sum of terms associated with edges and a sum of terms associated with all pairs of vertices. The coordinate Newton direction can be computed for each vertex in these models as well, and the stochastic coordinate descent approach can be applied to optimize the layout efficiently.

$$\underset{X \in \mathbb{R}^{2\times n}}{\text{minimize}} \quad \Phi_{\text{KK}}(X) \coloneqq \sum_{\substack{(i,j)\in V^2 \\ i\neq j}} \frac{(\|p_{ij}\| - l_{ij})^2}{2l_{ij}^2}$$

5 Numerical Experiment

In this section, we evaluate the proposed algorithm with various numerical experiments. We also confirm that the proposed algorithm efficiently provides a good initial placement even for larger weighted graphs.

5.1 Experimental Setup

We conducted all numerical experiments in this section using C ++17 compiled by GCC 10.5.0 on a laptop computer powered by Intel(R) Core(TM) i7-10510U CPU with 16 GB RAM.

For a fair comparison, we implemented the FR algorithm in C ++ based on NetworkX version 3.3 [15], SciPy 1.14.1 [31], and utilized the C ++ L-BFGS [25, 27] library for the L-BFGS algorithm. We also referred to the open-source code of the hexagonal grid [26] and Graphviz version 2.43.0 [9].

We used the 3×2 algorithms. As an initial placement, we used

- random initialization (no prefix),
- the SA initialization obtained by Simulated Annealing (SA-) [13],
- the proposed initialization obtained with coordinate Newton direction (CN-).

As an algorithm to solve the problem (1), we used

- FR algorithm (FR),
- L-BFGS algorithm (L-BFGS).

The key remark is that we slightly modified the FR algorithm so that it automatically adjust the stepsize. This is to isolate the effect of the proposed initialization from the effect of the stepsize. This is just done by a simple backtracking line search at the first iteration.

As parameters, we used initial temperature $t_0 = 1.5$ and the number of iterations $N_{\text{iter}}^{\text{CN}} = 2|V|^3/|E|$ for Algorithm 1, and we also set the total number of iterations of Simulated Annealing (SA) in Sec. 3.3 to the same value. Since the Algorithm 1 requires a computational time proportional to the degree of the selected vertex in each iteration, the

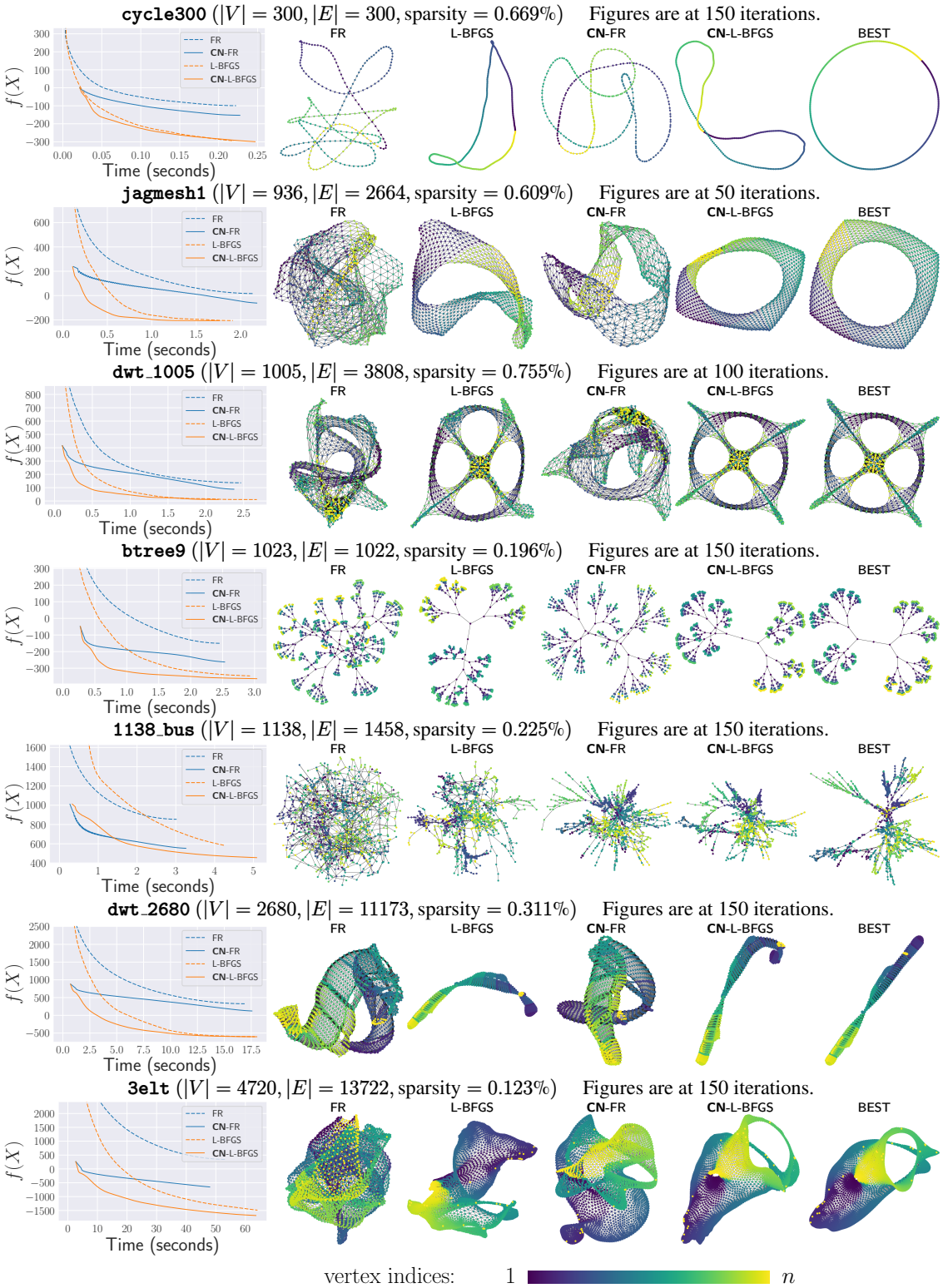

Figure 6: Experimental results. Our proposed method CN, Coordinate Newton, yields improved results for both FR and L-BFGS. The "BEST" reports the performance of CN-L-BFGS within at most 500 iterations. The color of vertices represents their indices.

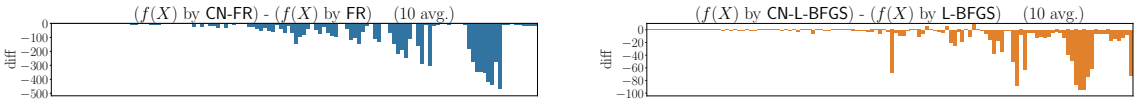

Figure 7: Comparison of the proposed initialization (CN) with random initialization (no prefix). In almost all cases, the difference is negative, meaning the proposed algorithm performed faster and better than random initialization.

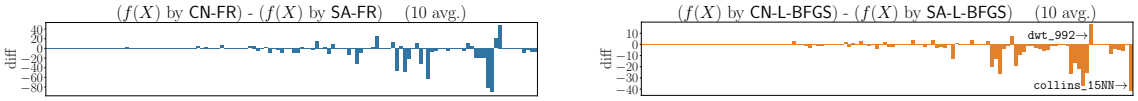

Figure 8: Comparison of the proposed initialization (CN) with SA initialization (SA) in Ref. [13]. In most cases, the proposed algorithm performed better than the SA initialization.

expected computational time per iteration is $\mathcal{O}(|E|/|V|)$. Consequently, we can roughly estimate that the total computational time of the proposed algorithm is $\mathcal{O}(|V|^2)$, equivalent to a few iterations of the FR or L-BFGS algorithms. All the codes are available at GitHub [16].

5.2 Plots and Visualizations

We first show the plots and visualizations of the algorithms in Fig. 6. The experiment details are as follows. We fixed the maximum number of iterations of the FR and L-BFGS algorithms as $N_{\text{iter}}^{\text{FR}} = N_{\text{iter}}^{\text{LBFGS}} = 200$. We tested with 7 graphs: `cycle300`, `jagmesh1`, `dwt_1005`, `btree9`, `1138_bus`, `dwt_2680`, and `3elt`. Here `cycle300` is a cycle graph with 300 vertices, and `btree9` is a perfect binary tree with $2^{9+1} - 1 = 1023$ vertices. Other graphs are from Sparse Matrix Collection [7], and these choices are based on Ref. [34].

In Fig. 6, the plots on the left illustrate the objective function values $\Phi(X)$, the average of the ten trials for each algorithm. The graphs on the right are at the iteration in which the most significant difference appeared among {50,100,150}-th iterations (or at the last iteration if it ended earlier). The vertices in the graphs are colored according to vertex indices.

The observations and implications of Fig. 6 are as follows. First, the plots with solid lines for CN generally demonstrate superior performance to those with dashed lines for non-CN. These results validate the efficacy of the proposed method. Secondly, the initial values of $\Phi(X)$ with CN-FR are significantly smaller than those with FR, suggesting that the proposed method provides a good initial placement. Thirdly, the visualization results also support the effectiveness of the proposed initial placement. In most cases, placements obtained with CN better reflect the nearly optimal arrangement shown in the "BEST".

As a side note, regardless of CN or non-CN, the algorithms with L-BFGS consistently outperform those with FR. This finding is consistent with prior research [19] and providing solid evidence for the effectiveness of L-BFGS.

5.3 Comparison with Other Initializations

Next, we conducted experiments to evaluate the performance of the proposed algorithm (CN) compared to random initialization (no prefix) and SA initialization (SA) with various graphs.

We used all undirected, connected, non-negative weighted graphs with 1,000 or fewer

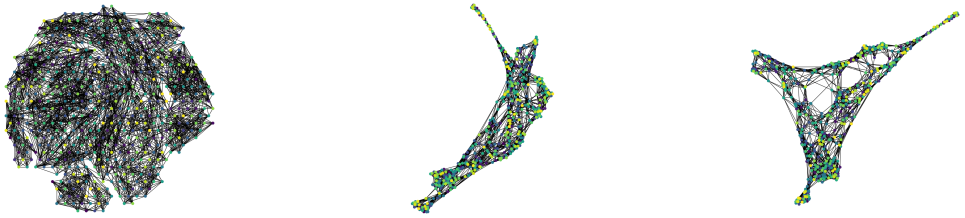

Figure 9: `Spectro_10NN`, where the proposed algorithm performed worse than random initialization. Left: Initial placement by CN. Middle and Right: 50th and 200th iteration of CN-L-BFGS.

Figure 10: An example of a case where $a_{i,j} \notin \{0, 1\}$. Left: the result of CN, which is better as the vertices are separated by colors. Right: the result of SA, which is worse as there is no separation.

vertices, which can be generated using the matrices from the Sparse Matrix Collection [7] as adjacency matrices. For fairness, when we compare with SA, we converted the graphs to an unweighted one. It means that we set weights $a_{i,j}$ to 1 if $a_{i,j} > 0$; otherwise, we set it to 0.

For the algorithms with random initial placement, we set $N_{\text{iter}}^{\mathsf{FR}} = N_{\text{iter}}^{\mathsf{LBFGS}} = 50$, the same as the default parameter in NetworkX [15]. For the CN algorithms, we set $N_{\text{iter}}^{\mathsf{FR}} = N_{\text{iter}}^{\mathsf{LBFGS}} = 45$, since the preprocessing step is expected to take as long as a few iterations of FR or L-BFGS, as mentioned in Sec. 5.1.

The results are shown in Figs. 7 and 8. The proposed algorithm performed better than random or SA initialization, except for a few cases. As for random initialization, one of such bad cases is `Spectro_10NN` shown in Fig. 9. The proposed algorithm failed to resolve the twist in the initial placement, shown in the figure, leading to a worse result. Still, the average performance of ten seeds for `Spectro_10NN` was almost the same as that of the random initialization. Fig. 11 provides examples for comparison with SA initialization. The proposed algorithm outperformed in `collins_15NN` and underperformed in `dwt_992`.

The interpretation of the results is as follows. First, results in Figs. 7 and 8 strongly support the effectiveness of the proposed algorithm. It successfully untangles the twists, leading to better outcomes with faster convergence. Even when the proposed algorithm performed worse, the difference was insignificant in almost all cases. Secondly, the structural difference between the initial placement and the optimal placement potentially affects the convergence speed. For example, the optimal shape of `collins_15NN` is linear, differing from a circle, resulting in SA's performance being inferior to the proposed algorithm, whereas the opposite holds for `dwt_992`. Although the proposed method utilizes a hexagonal lattice Q^{hex}, alternatives such as Q^{circle} may lead to improved performance of the proposed method in some cases.

Furthermore, although this is a case not considered in Ref. [13], we also conducted experiments with CN and SA for a case where the edge weight $a_{i,j}$ is not necessarily in $\{0, 1\}$. We generated a weighted graph with 100 vertices in three groups and 1000 edges randomly, and if the two vertices were in the same group, we set the edge weight to 1.0;

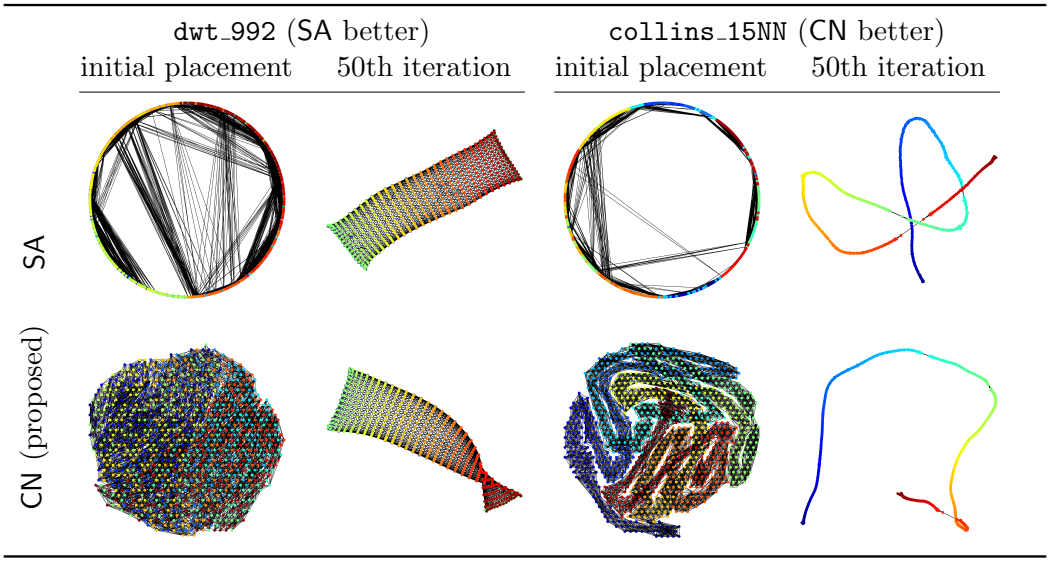

Figure 11: Visualization results showing initial and 50th iteration placements for `dwt_992` and `collins_15NN`.

otherwise, we set it to 0.1. It exhibits both strong and weak connections. Fig. 10 shows the difference between the two initializations. When we ignore the edge weights and just solve the problem (2), the graph is just an Erdős–Rényi graph, and thus SA cannot find any meaningful structure in the initial placement. In contrast, the proposed algorithm can identify the graph's structure, and we can observe that the left graph in Fig. 10 is separated into groups, as indicated by the node colors. This result suggests that our proposed algorithm is effective even for weighted graphs, extending the applicability of the preprocessing step.

6 Rationale for the Discretization Approach

This section explains why we did not apply the coordinate Newton direction technique directly to the original problem (1), but instead first transformed it into the discrete optimization problem (3). At first sight, one might expect that applying the coordinate Newton method directly to (1) would be more natural and possibly more efficient, since it avoids the detour through discretization. However, as we shall see, this approach suffers from intrinsic difficulties that undermine its effectiveness. By contrast, our discretization-based formulation allows us to circumvent these difficulties while still exploiting the advantages of the coordinate Newton direction. This not only justifies our methodological choice but also provides a foundation for exploring further optimization strategies built upon this framework. In what follows, we clarify the limitations of the direct approach and explain how our method addresses them.

6.1 Possible Alternatives

One natural idea for solving problem (1) is to apply existing optimization algorithms such as Stochastic Gradient Descent (SGD) or Stochastic Coordinate Descent.

Applying SGD to graph drawing, particularly to the Kamada–Kawai (KK) layout [21], has been a notable direction [1, 34]. The KK model determines the layout by minimizing a stress-like energy $\Phi_{i,j}^{\mathrm{KK}}$ defined for all vertex pairs. SGD corresponds to randomly selecting

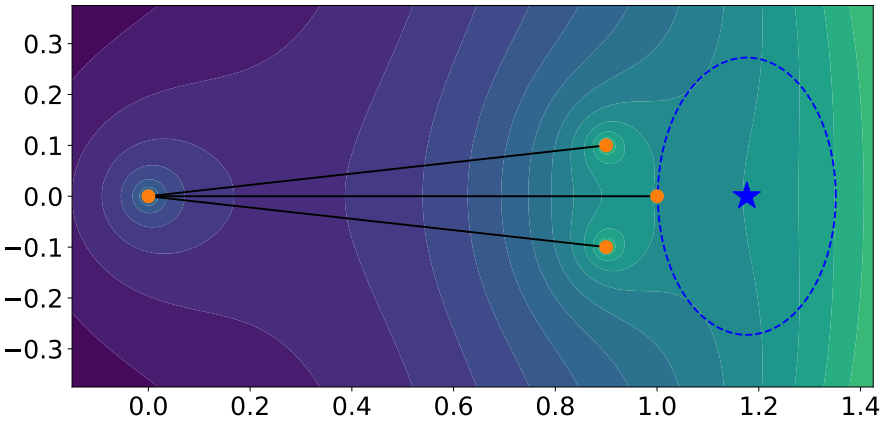

Figure 12: The inaccurate quadratic approximation. The blue star indicates the optimal solution for the quadratic approximation of $\Phi_1(x_1)$, but this differs significantly from the situation shown by the contour lines.

an edge i, j and updating x_i and x_j using the gradient of $\Phi_{i,j}^{\mathrm{KK}}$. This stochastic treatment reduces per-iteration complexity while retaining structural information, as $\Phi_{i,j}^{\mathrm{KK}}$ is defined in terms of graph distance.

In contrast, applying SGD directly to the FR model is ineffective. When $a_{i,j} = 0$, the energy term reduces to $\Phi_{i,j} = -k^2 \log d$, which depends only on Euclidean distance and ignores the graph structure. Consequently, gradients from a single sampled pair often fail to provide meaningful information for optimization.

Another approach is to use Stochastic Coordinate Descent, which resembles Randomized Subspace Newton methods [3, 11, 14, 18, 24]. For each vertex i, define the local energy

$$\Phi_i(x_i) \coloneqq \sum_{j \in V \setminus \{i\}} \Phi_{i,j}(\|x_i - x_j\|), \tag{6}$$

whose gradient (the net force on i) and Hessian are

$$\nabla \Phi_i(x_i) = \sum_{j \in V \setminus \{i\}} \left(\frac{a_{i,j}\|x_i - x_j\|}{k} - \frac{k^2}{\|x_i - x_j\|^2} \right) (x_i - x_j),$$

$$\nabla^2 \Phi_i(x_i) = \sum_{j \in V \setminus \{i\}} \left(\frac{a_{i,j}\|x_i - x_j\|}{k} - \frac{k^2}{\|x_i - x_j\|^2} \right) \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \sum_{j \in V \setminus \{i\}} \left(\frac{a_{i,j}}{k\|x_i - x_j\|} + \frac{2k^2}{\|x_i - x_j\|^4} \right) (x_i - x_j)(x_i - x_j)^\top.$$

A direct SCD scheme randomly selects a vertex i, applies Newton's method (or a regularized variant) to Φ_i using the above gradient and Hessian, and updates x_i iteratively until convergence.

Although reasonable in spirit, this approach is also ineffective in practice. In the following, we illustrate the reasons for this failure with nontrivial examples.

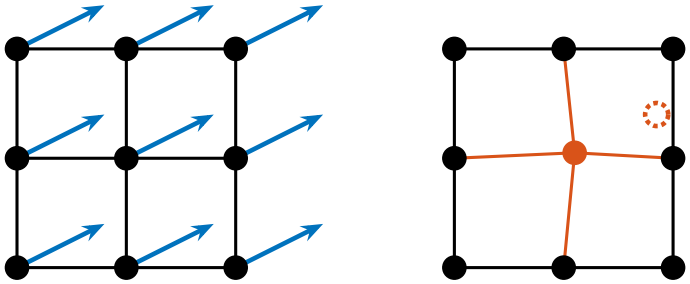

Figure 13: The blue arrows represent the acting forces in this situation. The center vertex exhibits only a minor shift in the coordinate Newton direction, while the red dashed vertex marks its ideal configuration.

6.2 Inaccuracy of Quadratic Approximation

One of the reasons it fails is the inaccuracy of the quadratic approximation; specifically, a particular issue arises when we restrict the optimization to a coordinate block.

Let G be a graph with $V = \{1, 2, 3, 4\}$ and $E = \{\{1, 2\}, \{2, 3\}, \{2, 4\}\}$. Set $k = 1/2$ and assign all positive edge weights $a_{i,j} = 1$ for every edge in E. The position of the vertices is

$$X = \begin{pmatrix} 1 & 0 & 0.9 & 0.9 \\ 0 & 0 & +0.1 & -0.1 \end{pmatrix}.$$

We show the contour of $\Phi_1(x_1)$ in Fig. 12. The key point of this example is that the Hessian

$$\nabla^2 \Phi_1(x_1) = \begin{pmatrix} 4.25 & 0 \\ 0 & 1.75 \end{pmatrix}$$

is positive definite and well-conditioned. Despite this favorable property of the Hessian, the coordinate Newton direction for x_1 results in a deviation from the global optimum. This issue arises from the inaccurate approximation of Φ_1 in the restricted block coordinates x_1. The attractive force from vertex 2 and the repulsive forces from vertices 3 and 4 cancel each other out, leading to a highly inaccurate quadratic approximation. This deficiency cannot be entirely resolved by modifying Newton's method, as it is an intrinsic and unavoidable limitation of the stochastic coordinate descent.

6.3 Ignorance of Other Vertices' Movements

Another reason is the ignorance of other vertices' movements when optimizing each vertex individually. When optimizing for a vertex i, the coordinate Newton direction treats all other vertices $V \setminus \{i\}$ as fixed.

Fig. 13 illustrates this issue. Consider a subset of vertices forming a mesh-like structure in G, where all vertices receive forces in the directions indicated by the blue arrows. In this situation, the FR and L-BFGS algorithms move all vertices simultaneously, allowing the simulation or optimization to proceed without issues. In contrast, the stochastic coordinate descent, as shown on the right side of the figure, brings stagnation. Here, all other vertices are considered fixed. Thus, optimizing the red vertex results in minimal movement, as its directly connected neighbors impede it. As a result, even after numerous iterations, little optimization is achieved. Thus, ignoring the movements of other vertices can be a significant limitation of the coordinate Newton method.

6.4 Rationale for Proposed Method

Our proposed method resolves issues in Secs. 6.2 and 6.3 by transforming the problem (1) into the problem (3) in Sec. 4.2, and optimizing Φ_i^{a} on Q^{hex}.

Regarding the issue in Sec. 6.2, this transformation brings the convexity of the objective function as it treats the repulsive forces via a hexagonal lattice, making the quadratic approximation more accurate than the original problem. Regarding the issue in Sec. 6.3, the discrete point set Q ensures that the stepsize is larger than ϵ, as each point is always separated by at least ϵ. This large stepsize prevents stagnation in the optimization, and helps explore the solution space more efficiently. Furthermore, this transformation also brings the benefit of reducing computational complexity from $\mathcal{O}(|V|^2)$ to $\mathcal{O}(|E|)$.

Thus, converting the problem is crucial to provide a high-quality initial placement with the coordinate Newton direction. There may also be better transformation methods, and further exploration is warranted.

7 Conclusion

In this study, we proposed a new initial placement with the coordinate Newton direction for the FR force model, the problem (1). The obtained initial placements have fewer twists than the random initialization, leading to faster convergence and better visualization. Numerical experiments revealed that the proposed method is effective across various graphs, extending the applicability of the preprocessing step. The proposed method may advance the graph drawing, and we also hope it highlights the potential of the stochastic coordinate descent and its variants for addressing a broader range of graph-related optimization problems.

Acknowledgment

We want to express our sincere gratitude to PL Poirion and Andi Han for their insightful discussions, which have greatly inspired and influenced this research. We also thank the developers of NetworkX and Graphviz. Their excellent work has been a great help in conducting this research.

This work was partially supported by JSPS KAKENHI (26KJ0936, 23H03351, 24K23853) and JST ERATO (JPMJER1903).

References

[1] R. Ahmed, F. De Luca, S. Devkota, S. Kobourov, and M. Li. Multicriteria Scalable Graph Drawing via Stochastic Gradient Descent, (SGD)2. *IEEE transactions on visualization and computer graphics*, 28(6):2388–2399, June 2022. doi:10.1109/TVCG.2022.3155564.

[2] A. Arleo, S. Miksch, and D. Archambault. *A Multilevel Approach for Event-Based Dynamic Graph Drawing*. The Eurographics Association, 2021. doi:10.2312/evs.20211063.

[3] C. Cartis, J. Fowkes, and Z. Shao. Randomised subspace methods for non-convex optimization, with applications to nonlinear least-squares, Nov. 2022. URL: http://arxiv.org/abs/2211.09873, arXiv:2211.09873, doi:10.48550/arXiv.2211.09873.

[4] H.-C. Chang and L.-C. Wang. A Simple Proof of Thue's Theorem on Circle Packing, Sept. 2010. URL: http://arxiv.org/abs/1009.4322, arXiv:1009.4322, doi:10.48550/arXiv.1009.4322.

[5] S.-H. Cheong and Y.-W. Si. Snapshot Visualization of Complex Graphs with Force-Directed Algorithms. In *2018 IEEE International Conference on Big Knowledge (ICBK)*, pages 139–145, Jan. 2018. URL: https://ieeexplore.ieee.org/document/8588785/?arnumber=8588785, doi:10.1109/ICBK.2018.00026.

[6] G. Csardi and T. Nepusz. The igraph software package for complex network research. *InterJournal*, Complex Systems:1695, 2006. URL: https://igraph.org.

[7] T. A. Davis and Y. Hu. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.*, 38(1):1:1–1:25, Dec. 2011. URL: https://dl.acm.org/doi/10.1145/2049662.2049663, doi:10.1145/2049662.2049663.

[8] P. Eades. A heuristic for graph drawing. *Congressus numerantium*, 42(11):149–160, 1984.

[9] J. Ellson, E. Gansner, L. Koutsofios, S. C. North, and G. Woodhull. Graphviz—Open Source Graph Drawing Tools. In P. Mutzel, M. Jünger, and S. Leipert, editors, *Graph Drawing*, pages 483–484, Berlin, Heidelberg, 2002. Springer. doi:10.1007/3-540-45848-4_57.

[10] T. M. J. Fruchterman and E. M. Reingold. Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11):1129–1164, 1991. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.4380211102, doi:10.1002/spe.4380211102.

[11] T. Fuji, P.-L. Poirion, and A. Takeda. Theoretical analysis of the randomized subspace regularized Newton method for non-convex optimization. *Open Journal of Mathematical Optimization*, 6:1–35, Oct. 2025. URL: https://ojmo.centre-mersenne.org/articles/10.5802/ojmo.46/, doi:10.5802/ojmo.46.

[12] P. Gajdoš, T. Ježowicz, V. Uher, and P. Dohnálek. A parallel Fruchterman–Reingold algorithm optimized for fast visualization of large graphs and swarms of data. *Swarm and Evolutionary Computation*, 26:56–63, Feb. 2016. URL: https://www.sciencedirect.com/science/article/pii/S2210650215000644, doi:10.1016/j.swevo.2015.07.006.

[13] F. Ghassemi Toosi, N. S. Nikolov, and M. Eaton. Simulated Annealing as a Pre-Processing Step for Force-Directed Graph Drawing. In *Proceedings of the 2016 on Genetic and Evolutionary Computation Conference Companion*, GECCO '16 Companion, pages 997–1000, New York, NY, USA, July 2016. Association for Computing Machinery. URL: https://dl.acm.org/doi/10.1145/2908961.2931660, doi:10.1145/2908961.2931660.

[14] R. Gower, D. Kovalev, F. Lieder, and P. Richtarik. RSN: Randomized subspace newton. In H. Wallach, H. Larochelle, A. Beygelzimer, F. dAlché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper_files/paper/2019/file/bc6dc48b743dc5d013b1abaebd2faed2-Paper.pdf.

[15] A. A. Hagberg, D. A. Schult, and P. J. Swart. Exploring Network Structure, Dynamics, and Function using NetworkX. *Python in Science Conference*, May 2008. URL: https://proceedings.scipy.org/articles/TCWV9851, doi:10.25080/TCWV9851.

[16] H. Hamaguchi. Hirokihamaguchi/Initial_Placement_for_Fruchterman–Reingold_Force_Model_with_Coordinate_Newton_Direction, 2024. URL: https://github.com/HirokiHamaguchi/Initial_Placement_for_Fruchterman-Reingold_Force_Model_with_Coordinate_Newton_Direction.

[17] D. Harel and Y. Koren. A Fast Multi-Scale Method for Drawing Large Graphs. *Journal of Graph Algorithms and Applications*, 6(3):179–202, Jan. 2002. URL: https://jgaa.info/index.php/jgaa/article/view/paper51, doi:10.7155/jgaa.00051.

[18] R. Higuchi, P.-L. Poirion, and A. Takeda. Fast Convergence to Second-Order Stationary Point through Random Subspace Optimization, June 2024. URL: http://arxiv.org/abs/2406.14337, arXiv:2406.14337, doi:10.48550/arXiv.2406.14337.

[19] H. Hosobe. Numerical optimization-based graph drawing revisited. In *2012 IEEE Pacific Visualization Symposium*, pages 81–88, 2012. doi:10.1109/PacificVis.2012.6183577.

[20] Y. Hu. Efficient, high-quality force-directed graph drawing. *The Mathematica journal*, 10:37–71, 2006. URL: https://api.semanticscholar.org/CorpusID:14599587.

[21] T. Kamada and S. Kawai. An algorithm for drawing general undirected graphs. *Information Processing Letters*, 31(1):7–15, Apr. 1989. URL: https://www.sciencedirect.com/science/article/pii/0020019089901026, doi:10.1016/0020-0190(89)90102-6.

[22] J. Li, Y. Tao, K. Yuan, R. Tang, Z. Hu, W. Yan, and S. Liu. Fruchterman–reingold hexagon empowered node deployment in wireless sensor network application. *Sensors*, 22(5179), 2022. URL: https://www.mdpi.com/1424-8220/22/14/5179, doi:10.3390/s22145179.

[23] D. C. Liu and J. Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45(1):503–528, Aug. 1989. doi:10.1007/BF01589116.

[24] R. Nozawa, P.-L. Poirion, and A. Takeda. Randomized subspace gradient method for constrained optimization, July 2023. URL: http://arxiv.org/abs/2307.03335, arXiv:2307.03335, doi:10.48550/arXiv.2307.03335.

[25] N. Okazaki. Chokkan/liblbfgs, Oct. 2024. URL: https://github.com/chokkan/liblbfgs.

[26] A. J. Patel. Hexagonal Grids. Technical report, Red Blob Games, 2013. URL: https://www.redblobgames.com/grids/hexagons/.

[27] Y. Qiu. Yixuan/LBFGSpp, Oct. 2024. URL: https://github.com/yixuan/LBFGSpp.

[28] M. Tiezzi, G. Ciravegna, and M. Gori. Graph Neural Networks for Graph Drawing. *IEEE Transactions on Neural Networks and Learning Systems*, 35(4):4668–4681, Apr. 2024. URL: http://arxiv.org/abs/2109.10061, arXiv:2109.10061, doi:10.1109/TNNLS.2022.3184967.

[29] D. Tunkelang. *A Numerical Optimization Approach to General Graph Drawing*. PhD thesis, Carnegie Mellon University, 1999.

[30] T. L. Veldhuizen. Dynamic Multilevel Graph Visualization, Dec. 2007. URL: http://arxiv.org/abs/0712.1549, arXiv:0712.1549, doi:10.48550/arXiv.0712.1549.

[31] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi:10.1038/s41592-019-0686-2.

[32] C. Walshaw. A Multilevel Algorithm for Force-Directed Graph-Drawing. *Journal of Graph Algorithms and Applications*, 7(3):253–285, Jan. 2003. URL: https://jgaa.info/index.php/jgaa/article/view/paper70, doi:10.7155/jgaa.00070.

[33] S. Wright and B. Recht. *Optimization for Data Analysis*. Cambridge University Press, 2022. URL: https://books.google.co.jp/books?id=lOViEAAAQBAJ.

[34] J. X. Zheng, S. Pawar, and D. F. M. Goodman. Graph Drawing by Stochastic Gradient Descent. *IEEE Transactions on Visualization and Computer Graphics*, 25(9):2738–2748, Sept. 2019. URL: https://ieeexplore.ieee.org/document/8419285, doi:10.1109/TVCG.2018.2859997.

A NetworkX Implementation

As supplementary information, this section explains how to solve the problem (1) and obtain the final placement. At the time of writing, NetworkX, an open-source library for graph analysis in Python, is set to release the L-BFGS-based method that we have implemented. Although this method does not use our proposed initial placement, we provide some implementation details as a reference in Sec. A.3.

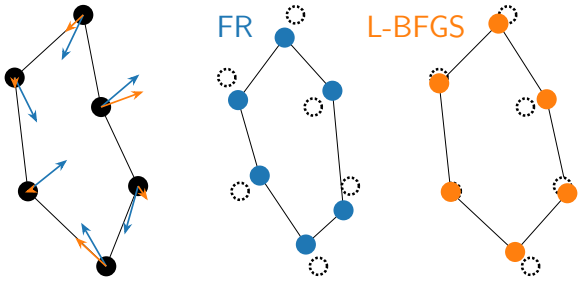

Figure 14: Comparison of the FR and L-BFGS algorithms. Blue arrows represent fixed stepsizes in FR, whereas orange arrows represent adaptive stepsizes determined by the approximate inverse Hessian

A.1 Fruchterman–Reingold Algorithm

Algorithm 2: Fruchterman–Reingold algorithm

Input: Graph $G = (V, E)$, Weights $(a_{i,j})_{\{i,j\}\in E}$, Parameters $N_{\text{iter}}^{\mathsf{FR}} \in \mathbb{N}$, $t_0 > 0$, and Initial placement $X = (x_1, \dots, x_n)$

Output: Final placement X

$t \leftarrow t_0$;
for $m \leftarrow 1$ **to** $N_{\text{iter}}^{\mathsf{FR}}$ **do**
  compute gradient $\nabla\Phi_i(x_i)$ for all $i \in V$;
  $x_i^{\text{new}} \leftarrow x_i - t\frac{\nabla\Phi_i(x_i)}{\|\nabla f_i(x_i)\|}$ for all $i \in V$;
  $x_i \leftarrow x_i^{\text{new}}$;
  $t \leftarrow t - t_0/N_{\text{iter}}^{\mathsf{FR}}$;
  if *convergence condition is satisfied* **then**
    break;
return X;

The Fruchterman–Reingold algorithm [10] is the original and most standard approach to obtain the equilibrium state of the force model. The pseudo-code is shown in Algorithm 2, which is a variant of the gradient (steepest) descent method with the function Φ_i in Eq. (6) [29].

Algorithm 2 is based on the original pseudo-code [10] and implementation in NetworkX [15] with some omitted details. We typically draw the initial placement X from a uniform distribution on $[0, 1]^{2\times n}$. The parameter t denotes the temperature, which governs the stepsize along the steepest descent. As the temperature gradually decreases, the algorithm converges to a particular placement, which is not necessarily a local optimum of the problem (1).

A.2 L-BFGS Algorithm

Another approach is to leverage the L-BFGS algorithm [19]. Using only a few recent gradient vectors, the L-BFGS algorithm approximates the inverse Hessian of the objective function Φ [23]. L-BFGS is known to be very efficient for large-scale optimization problems. We can apply the L-BFGS algorithm to the optimization problem (1) via flattening the matrix $X \in \mathbb{R}^{2\times n}$ to a vector $\overline{X} \in \mathbb{R}^{2n}$. Refer to Fig. 14 for a comparison to the FR algorithm.

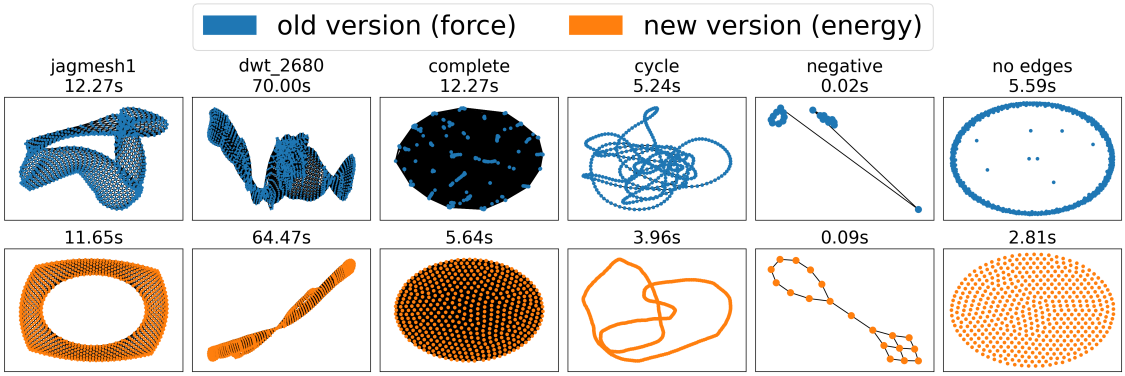

Figure 15: The FR algorithm and L-BFGS algorithm in NetworkX.

A.3 NetworkX Implementation

In this subsection, we explain the details of our NetworkX implementation. Refer to Fig. 15 for a comparison to the FR algorithm. The problem setting considered in NetworkX is similar to that described in Sec. 2, but there are some key differences. First, the number of dimensions is not always two. Since the extension of the method is straightforward, we restrict our discussion to two-dimensional layouts. Other than that, the following differences exist:

- Negative edge weights are allowed.
- Directed graphs are supported, not just undirected graphs.
- The graph is not always connected.

Under these settings, the problem (1) can be unbounded. It means that the optimal solution does not exist in a finite region. When edge weights are negative or the graph is disconnected, i.e., when it consists of $m > 1$ connected components ($V = \bigoplus_{j=1}^{m} V_j$), the solution may be unbounded, since the farther apart the vertices are, the lower the energy.

To overcome this issue, we do the following suitable modifications. First, when the graph has negative edge weights, one may transform them into non-negative values, for example, by taking their absolute values. Second, when the graph is a directed one, only the sum $a_{i,j} + a_{j,i}$ matters in the formulation, so symmetrization is sufficient. Thus, the directed and possibly negatively weighted adjacency matrix $\tilde{A} = (\tilde{a}_{i,j}) \in \mathbb{R}^{n \times n}$ can be transformed into a symmetric matrix $A = (a_{i,j})$, where $a_{i,j} = (|\tilde{a}_{i,j}| + |\tilde{a}_{j,i}|)/2 \geq 0$. Third, we add additional forces to the energy function to prevent the divergence of each connected component. In general, adding terms that attract each group to the center is a common approach to avoid unboundedness. We adopt this approach with the center $x_{\text{center}} = (0.5, 0.5)^\top$, as it is the center of the expected layout bounding box.

Based on Sec. 2, let us define the sum of terms associated with the vertex x_i be Φ_i, which is explicitly given by

$$\Phi_i(x_i) \coloneqq \sum_{j \in V \setminus \{i\}} \left(\Phi_{i,j}(\|x_i - x_j\|) + \Phi_{j,i}(\|x_j - x_i\|) \right).$$

The i-th row of the gradient $\nabla\Phi(X) \in \mathbb{R}^{2 \times n}$ is

$$(\nabla\Phi(X))_i^\top = \nabla\Phi_i(x_i) = \sum_{j \in V \setminus \{i\}} \left(\frac{(a_{i,j} + a_{j,i})\|x_i - x_j\|}{k} - \frac{2k^2}{\|x_i - x_j\|^2} \right) (x_i - x_j)$$

$$= 2 \sum_{j \in V \setminus \{i\}} \left(\frac{a_{i,j} \|x_i - x_j\|}{k} - \frac{k^2}{\|x_i - x_j\|^2} \right) (x_i - x_j).$$

To obtain the final layout while preventing the divergence, we need to minimize $\Phi(X) + f_{\mathrm{g}}(X)$, where $f_{\mathrm{g}}(X)$ is the additional term. Let the center of gravity of the j-th connected component V_j $(1 \leq j \leq m)$ be $g_j \coloneqq \frac{1}{|V_j|} \sum_{i \in V_j} x_i$. Then, using some constant $c_{\mathrm{g}} > 0$, we define the additional force as

$$\Phi_{\mathrm{g}}(X) \coloneqq c_{\mathrm{g}} \sum_{j=1}^{m} \frac{|V_j|}{2} \|g_j - x_{\mathrm{center}}\|_2^2,$$

with the gradient

$$(\nabla \Phi_{\mathrm{g}}(X))_i = c_{\mathrm{g}}(g_j - x_{\mathrm{center}}) \quad \text{where} \quad i \in V_j.$$

We can now apply the L-BFGS algorithm to these functions to derive the algorithm. Further implementation details and experimental results are available in our merged pull request to the NetworkX.